

Curso Introducción a la Estadística con R

Clase 3: Introducción a la programación en R

Directorios de trabajo y proyectos de R (.Rproj)

Directorios de trabajo

- Para abrir en R un archivo guardado en tu computadora, debes especificar en qué carpeta está guardado, para esto hay varias opciones. Primero, puedes fijar un directorio por defecto:

```
# Puedo fijar el directorio de trabajo con la función setwd()
# Fijar la carpeta donde vamos a importar y exportar los archivos:
setwd("micompu/micarpeta")
getwd() # Con esta función puedo consultar el directorio
```

```
# Ahora, si quiero leer un archivo que esté en "micompu/micarpeta" simplemente
# escribo su nombre dentro de la función, en el lugar del "path".
```

```
# Supongamos que tengo dentro de la carpeta "micarpeta" un excel con datos
# de desempleo en Uruguay:
```

```
library(readxl)
desempleo_uru <- read_excel("data/desempleo.xlsx")
head(desempleo_uru, 4)
```

```
## # A tibble: 4 × 2
##   Year  tasa
##   <dbl> <dbl>
## 1  1990   8.5
## 2  1991   8.9
## 3  1992    9
## 4  1993   8.3
```

Directorios de trabajo

También podemos no fijar un directorio para la sesión e ir especificando los directorios completos dentro de cada función:

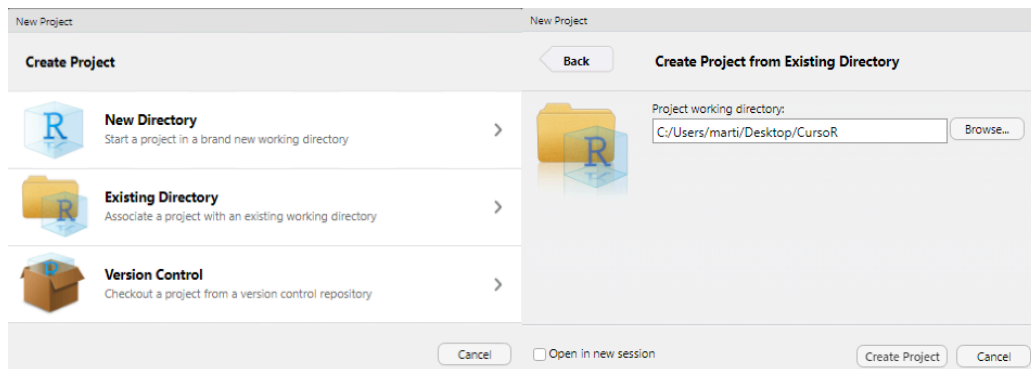
```
desempleo_uuru <- read_excel("micompu/micarpeta/data/desempleo.xlsx")
```

Proyectos de R (.Rproj)

- La mejor práctica para que nuestros scripts sean portables y reproducibles, es utilizar R Projects (.Rproj).
- Para crear un .Rproj vamos a `File/New Project` y ahí nos encontramos con la opción de crear una carpeta para guardar los archivos o utilizar una carpeta ya existente.
- Al crear un proyecto de R se creará un archivo de extensión .Rproj, cuando le damos click se inicia una nueva sesión de R cuyo directorio es por defecto la carpeta en la que está guardado.
- Podemos usar directorios relativos dentro de la carpeta en la que se aloja nuestro .Rproj para importar y exportar datos a y desde R. Esto hace que uno pueda cambiar la carpeta o compartirla y el script correrá de igual manera (a diferencia de si utilizamos `setwd()`)

Proyectos de R (.Rproj)

1. Creen una carpeta para guardar el material del curso
2. Descarga del EVA del curso las carpetas "Data" y las carpetas "scripts" de los días 1 y 2. Pegalas en la carpeta del punto 1 (guarda todos los scripts dentro de la misma carpeta)
3. Crea además dos carpetas más dentro de tu carpeta: "graficos" y "resultados"
4. Abran RStudio y desde ahí creen un *.Rproj*, desde existing directory, seleccionando la carpeta del punto 1



Proyectos de R (.Rproj)

5. Deberían ver en su carpeta algo así:

> CursorR

☐ Name	Date modified	Type	Size
 data	15/07/2021 18:06	File folder	
 graficos	15/07/2021 18:06	File folder	
 resultados	15/07/2021 18:06	File folder	
 scripts	15/07/2021 18:06	File folder	
 CursorR	15/07/2021 18:07	R Project	1 KB

Proyectos de R (.Rproj)

6. Abran el archivo `.Rproj` y desde ahí usando File/Open File abren los scripts dentro de la carpeta "scripts". Es importante que los abran desde la sesión que inicia el proyecto y no directamente haciendo click en el script.
7. Ya estamos listos para empezar!

Dialectos

Ejercicio

Supongamos que tengo estos datos:

```
## — Attaching core tidyverse packages ————— tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.5
## ✓ forcats    1.0.1      ✓ stringr   1.6.0
## ✓ ggplot2    4.0.2      ✓ tibble    3.2.1
## ✓ lubridate  1.9.4      ✓ tidyr     1.3.1
## ✓ purrr      1.0.4
## — Conflicts ————— tidyverse_conflicts() —
## ✗ dplyr::filter()      masks stats::filter()
## ✗ dplyr::group_rows()  masks kableExtra::group_rows()
## ✗ dplyr::lag()          masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
##
## Attaching package: 'data.table'
##
##
## The following objects are masked from 'package:lubridate':
##
##   hour, isoweek, mday, minute, month, quarter, second, wday, week,
##   yday, year
##
## The following objects are masked from 'package:dplyr':
##
##   between, first, last
##
## The following object is masked from 'package:purrr':
##
##   transpose
```

Ejercicio

Supongamos que tengo estos datos:

data

```
## # A tibble: 35 × 5
##   year      gdp_lcu inflation unemployment presidente
##   <dbl>      <dbl>    <dbl>         <dbl> <chr>
## 1  1985 249277574100    72.2           NA Sanguinetti
## 2  1986 271238450500    76.4           NA Sanguinetti
## 3  1987 292918912000    63.6           NA Sanguinetti
## 4  1988 297256857900    62.2           NA Sanguinetti
## 5  1989 300538279400    80.4           NA Sanguinetti
## 6  1990 301431925100   113.           NA Lacalle
## 7  1991 312099023700   102.           9.01 Lacalle
## 8  1992 336853433700    68.5           8.98 Lacalle
## 9  1993 345805469000    54.1           8.94 Lacalle
## 10 1994 370984750100    44.7           9 Lacalle
## # i 25 more rows
```

Ejercicio

¿Qué quiero hacer con el código debajo?

```
aggregate(  
  data[data$presidente %in% c("Vázquez", "Sanguinetti"),  
        c("gdp_lcu", "inflation")],  
  by = list(presidente = data$presidente[data$presidente %in% c("Vázquez", "Sanguinetti")]),  
  FUN = mean,  
  na.rm = TRUE  
)
```

Ejercicio

¿Qué quiero hacer con el código debajo?

```
data_dt <- data
setDT(data_dt)
data_dt[presidente %in% c("Vázquez", "Sanguinetti"),
        c("presidente", "gdp_lcu", "inflation") ][
        , lapply(.SD, mean), by = presidente]
```

Ejercicio

¿Qué quiero hacer con el código debajo?

```
data %>%  
  filter(presidente %in% c("Vázquez", "Sanguinetti")) %>%  
  select(presidente, gdp_lcu, inflation) %>%  
  group_by(presidente) %>%  
  summarise_all(mean)
```

```
aggregate(  
  data[data$presidente %in% c("Vázquez", "Sanguinetti"),  
        c("gdp_lcu", "inflation")],  
  by = list(presidente = data$presidente[data$presidente %in% c("Vázquez", "Sanguinetti")]),  
  FUN = mean,  
  na.rm = TRUE  
)
```

```
##      presidente      gdp_lcu inflation  
## 1 Sanguinetti 344923753631 46.168819  
## 2      Vázquez 584455715198  7.416316
```

Data.table

```
data_dt <- data
setDT(data_dt)
data_dt[presidente %in% c("Vázquez", "Sanguinetti"),
        c("presidente", "gdp_lcu", "inflation") ][
        , lapply(.SD, mean), by = presidente]
```

```
##      presidente      gdp_lcu inflation
##      <char>      <num>      <num>
## 1: Sanguinetti 344923753631 46.168819
## 2:      Vázquez 584455715198  7.416316
```



```
data %>%  
  filter(presidente %in% c("Vázquez", "Sanguinetti")) %>%  
  select(presidente, gdp_lcu, inflation) %>%  
  group_by(presidente) %>%  
  summarise_all(mean)
```

```
## # A tibble: 2 × 3  
##   presidente      gdp_lcu inflation  
##   <chr>          <dbl>    <dbl>  
## 1 Sanguinetti 344923753631.    46.2  
## 2 Vázquez    584455715198.     7.42
```

Dialectos

- En R podemos realizar una misma operación de muchas maneras distintas. Puesto de otra manera, R como lenguaje de programación tiene distintos "dialectos", esto es, paquetes (o conjuntos de paquetes) con sus propias funciones, sintaxis y comunidad de usuarios.
- Para la mayoría de las funciones requeridas para un análisis de datos estándar (importar datos, manipular, modelar y visualizar) existen -de forma muy simplificada- tres grandes dialectos: **R Base**, **tidyverse** y **data.table**.
- Tidyverse es una colección de paquetes diseñados para el análisis de datos. Este conjunto de paquetes comparte una filosofía de diseño, gramática y estructura de datos.
- Las ventajas de Tidyverse están en su gramática (fácil de leer lo que invita a compartir y replicar), consistencia, alcance y su numerosa y creciente comunidad.

Dialectos (ejemplo)

```
encuesta # Retomemos el data.frame "encuesta"
```

```
##   edad ideologia      voto  
## 1   18 Izquierda Partido A  
## 2   24 Izquierda Partido A  
## 3   80  Derecha Partido C
```

```
# Supongamos que quiero quedarme solo con las variables de edad y voto
```

```
encuesta_base <- encuesta[ , c("edad", "voto")] # R Base  
colnames(encuesta_base)
```

```
## [1] "edad" "voto"
```

```
encuesta_dt <- as.data.table(encuesta)[ , .(edad, voto)] # Datatable  
colnames(encuesta_dt)
```

```
## [1] "edad" "voto"
```

```
encuesta_tidy <- select(encuesta, edad, voto) # Tidyverse  
colnames(encuesta_tidy)
```

```
## [1] "edad" "voto"
```

Tidyverse

Tidyverse

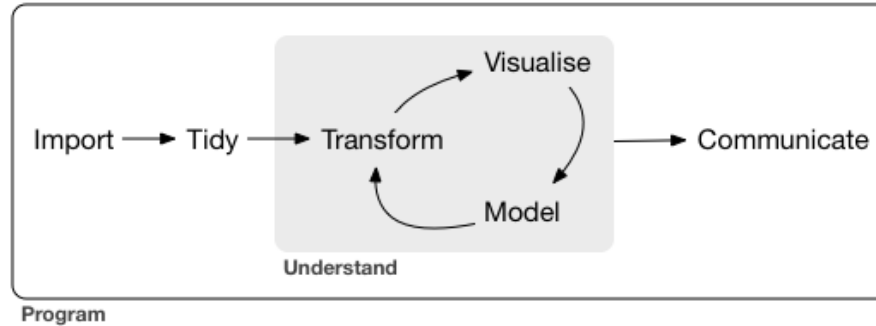
Tidyverse cuenta con varios paquetes que sirven para distintos tipos de tareas específicas. Podemos cargar todos los paquetes de forma conjunta:

```
# install.packages("tidyverse")  
library(tidyverse)
```



Tidyverse

La mejor manera de entender los principios de tidyverse es a través del libro del creador de tidyverse (Hadley Wickham) y Garrett Grolemund "R for Data Science" (2018).



Tidyverse: paquetes

Paquetes que Tidyverse carga:

- **readr**: importar y exportar datos
- **dplyr**: manipulación de datos
- **tidyr**: manipulación de datos
- **ggplot2**: visualización de datos
- **purrr**: programación avanzada
- **tibble**: estructura de datos
- **forcats**: factores
- **stringr**: variables de caracteres



Tidyverse: paquetes

Estos son algunos paquetes (para tareas más específicas) que forman parte del Tidyverse pero se tienen que cargar por separado:

- `readxl`: importar datos (excel)
- `haven`: importar (Stata, SPSS, SAS)
- `lubridate`: manipulación de fechas
- `rvest`: webscrapping
- `glue`: combinar data
- `tidymodels`: modelar datos

Importar y exportar datos

Importar datos

- Hasta ahora trabajamos principalmente con datos ingresados manualmente con las funciones `c()` y `data.frame()`
- Normalmente cuando trabajamos con datos solemos utilizar datos ya creados guardados en los formatos de otros programas (ej. Excel, Stata, SPSS)
- Existen varios paquetes que permiten importar y exportar datos desde distintos formatos. Algunos de los más utilizados son
 - `readr` para archivos csv
 - `haven` para SPSS; STATA
 - `readxl` para excel

Importar datos

Distintas funciones nos sirven para importar datos a R desde distintos formatos. Veamos algunos ejemplos:

```
# Con la función read_csv() del paquete readr importamos archivos .csv
library(tidyverse)
gapminder_csv <- read_csv("data/gapminder.csv")

# Con la función read_excel() del paquete readxl importamos archivos excel
library(readxl)
gapminder_excel <- read_excel("data/gapminder.xlsx")
```

```
# Vemos que los dataframes son iguales, tienen la mismas filas y columnas
dim(gapminder_csv)
```

```
## [1] 1704    6
```

```
dim(gapminder_excel)
```

```
## [1] 1704    6
```

Importar datos

Algunos paquetes incluyen datos, por ejemplo, gapminder. En la documentación del paquete se encuentra el nombre de los datos. Con una simple asignación los podemos cargar

```
library(gapminder)

data_gapminder <- gapminder
head(data_gapminder)
```

```
## # A tibble: 6 × 6
##   country      continent  year lifeExp      pop gdpPercap
##   <fct>        <fct>    <int>  <dbl>    <int>    <dbl>
## 1 Afghanistan Asia      1952   28.8  8425333    779.
## 2 Afghanistan Asia      1957   30.3  9240934    821.
## 3 Afghanistan Asia      1962   32.0 10267083    853.
## 4 Afghanistan Asia      1967   34.0 11537966    836.
## 5 Afghanistan Asia      1972   36.1 13079460    740.
## 6 Afghanistan Asia      1977   38.4 14880372    786.
```

Importar datos

También es posible importar datos guardados en los formatos de otros softwares estadísticos como SPSS o Stata. Para esto usaremos el paquete haven.

```
library(haven)

# SPSS
gapminder_spss <- read_spss("data/gapminder.sav")

# STATA
gapminder_stata <- read_stata("data/gapminder.dta")
```

O podríamos llamar a la función y paquete dado que generalmente solo utilizamos una función de los paquetes que cargan datos (depende del caso obviamente)

```
# SPSS
gapminder_spss <- haven::read_spss("data/gapminder.sav")

# STATA
gapminder_stata <- haven::read_stata("data/gapminder.dta")
```

Importar datos

R también cuenta con sus propios formatos de almacenamiento de datos (`.rds` y `.Rdata` o `.rda`). Este enfoque es poco práctico si queremos usar los datos almacenados en otro programa, pero muy útil si solamente usaremos R dado que mantiene la información tal cual estaba en R (por ej. tipos de variables o atributos):

```
# Para esto no necesitamos cargar paquetes.
# Guardar un objeto como .rds:
saveRDS(object = data_gapminder,
        file = "resultados/data_gapminder.rds")

# Leemos un archivo .rds
miobjeto_rds <- readRDS(file = "resultados/data_gapminder.rds")

# Con .rda se pueden guardar varios objetos al mismo tiempo!
# Exportamos un archivo .Rdata
save(data_gapminder, miobjeto_rds,
     file = "resultados/dos_dataframes.Rdata")

# Importamos un archivo .Rdata
load("resultados/dos_dataframes.Rdata")
```

Exportar datos

- También podemos guardar archivos desde R en otros formatos.
- Con `readr` podemos exportar archivos en formato .csv
- Con `writexl` podemos exportar directamente un excel.
- Con `haven` podemos exportar archivos en formato .dta (Stata) y .sav (SPSS)

Importar y exportar datos

Argumentos a tener en cuenta:

- **Nombre de columnas:** a veces debemos especificar si queremos que la primera fila de nuestros datos sean el nombre de las variables
- **Nombre de filas:** de igual manera, a veces podemos especificar si queremos que la primera columna sea el nombre de las filas (sirve para identificadores de caso por ej.)
- **Etiquetas de variables:** cuando los datos que queremos importar tienen etiquetas (pasa mucho en encuestas) podemos cargarlas como etiquetas o cargar solamente la etiqueta como cadena o factores. Ver capítulo 4 de Urdinez, F. & Labrin, A. (Eds.) (2020)
- **Append:** algunas funciones permiten agregar filas debajo de un archivo (esto es muy útil para ir actualizando bases de datos)

Ejercicio

- (1) En la carpeta data encontrarán un archivo excel llamado "urudata_sheets", leer el archivo en R*
- (2) Leer la segunda hoja del archivo*